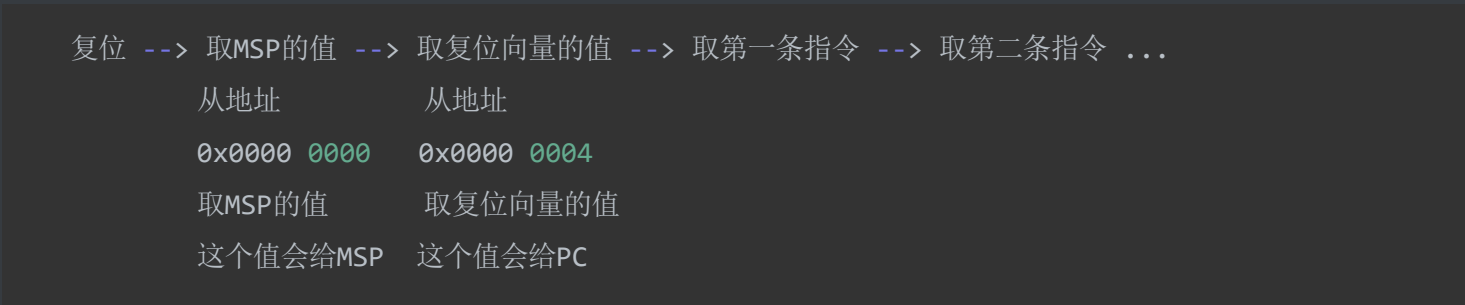


Arm启动

Cortex-M的启动流程

以下流程来自《The Definitive Guide to Arm Cortex-M3 and Cortex-M4 Processors_c》



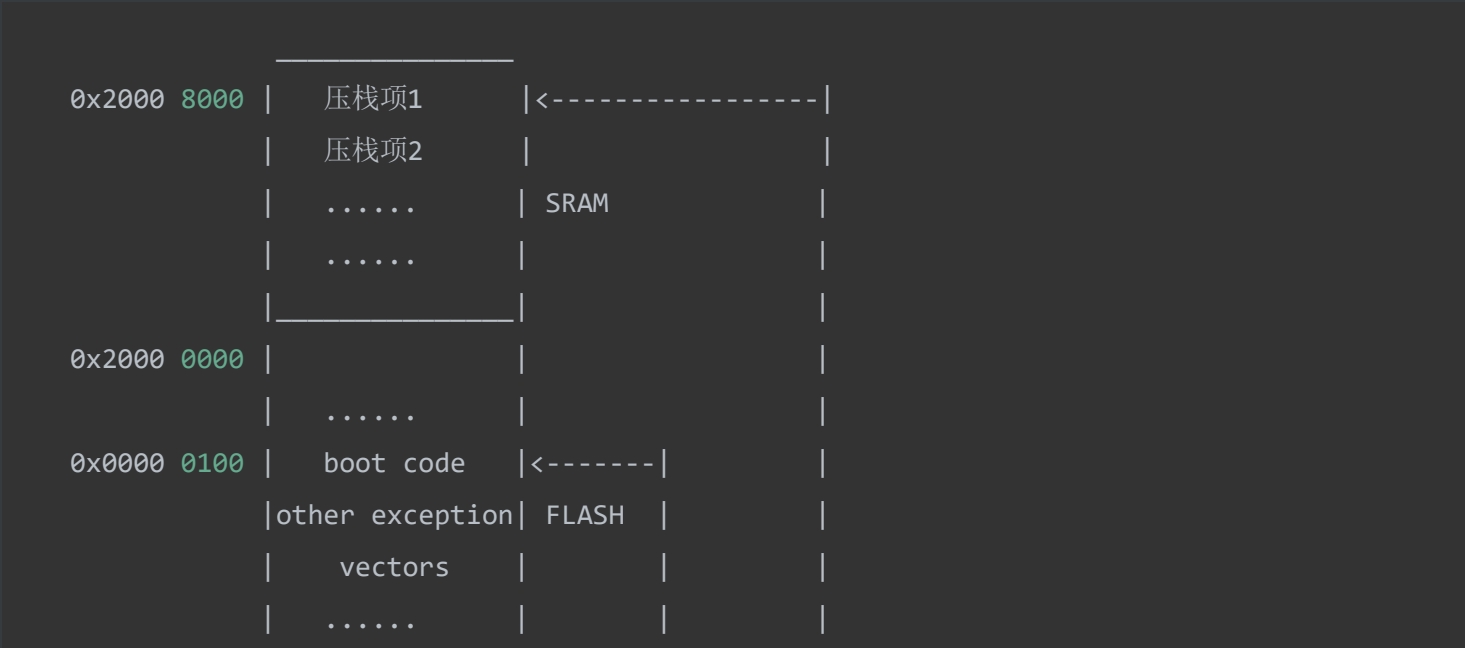
MSP是 main stack pointer 主栈指针

复位向量 这个东西是上电之后要执行的代码所在的地址，reboot之后第一行代码执行什么就由它决定

PC是程序计数器，它本质上是一个寄存器，记录了接下来要执行的代码的地址

结合Cortex-M的内存模型进行更详细的说明

Cortex-M的内存模型



```
0x0000 0004 | 0x0000 0101 |-----|
0x0000 0000 | 0x2000 8000 |-----|
```

解释几个问题：

1. 为什么0x0000 0004里的内容明明时0x0000 0101，但是却指向了0x0000 0100？

因为0x0000 0101并不是一个数值，而是指向一个“函数”，0x0000 0101中最后那个1，意思是0x0000 0100的内容要以thumb指令集进行执行，这个“1”又叫做T=1

而0x0000 0000中的内容并不指向函数，0x2000 8000就是一个栈地址，这个地址会赋值给MSP，因此它是一个单纯的“数值”

2. FLASH，SRAM是什么？

FLASH和SRAM都是存储设备，以STM32为例，他们都在STM32片内，但是不在Cortex-M核心里，站在Cortex-M的角度看，FLASH和SRAM都是外部存储设备，站在STM32的角度看，STM32集成了Cortex-M核心、FLASH、SRAM；

FLASH只允许核心读，想要写FLASH必须通过FLASH控制器进行，也就是我们说的烧录器。

FLASH中的内容掉电不会消失；

SRAM更像电脑的运行内存，数据掉电就消失，但是SRAM的访问速度要比FLASH快；

3. 上电之后，内存里的程序以什么顺序执行？

首先，从0x0000 0000 读值，赋值给MSP

然后，从0x0000 0004 读值，赋值给PC

再然后PC就可以接管程序执行的顺序了

4. 请注意！上面这个图是Cortex-M给的内存模型，SRAM从0x2000 0000 到 0x2000 8000 一共是32K，但实际上STM32F103C8的SRAM只有20K，不同型号的MCU这个值可能都是不一样的，不要生搬硬套。

一个简单的启动代码

先搞一个简单的demo代码：

```
# 汇编代码
```

```

.syntax unified
.cpu cortex-m3
.thumb

.section .isr_vector, "a"
.word 0x20008000
.word boot_code

.global boot_code
.type boot_code, %function

boot_code:
    mov r1, #0xA5
    b .

```

我看的课程使用STM32F103C8，这个型号的soc手册上说明了这款mcu有三种启动方式，根据BOOT1和BOOT0的电平状态，分别可以从Main Flash memory、system memory、embedded SRAM进行启动，一般的启动方式就是从Main Flash memory启动；

STM32F103C8手册上，对从Main Flash memory启动做了如下描述：

可以从0x0000 0000或0x800 0000启动；（但是实际上，STM32在跳线到使用Flash启动时，会把0x0000 0000映射到0x0800 0000上）

也就是说，把代码放在0x800 0000能通过第一种方式启动，但是放在其他位置不行；这是由ST公司设计的

因此需要写链接脚本，指定代码存放的位置

```

ENTRY(boot_code) # 指定程序入口，ENTRY对真实的cpu复位流程没有影响
                  # ENTRY主要是写给仿真程序看的
                  # 真实代码里没有这个ENTRY也不影响。

SECTIONS
{
    . = 0x8000000; # "."类似于光标，移动到哪里，就从哪里开始写
                  # 之后的操作会从0x800 0000为基准进行
    .text : { *(.isr_vector)} # 指定 将所有.o文件里的.isr_vector输出到.text里
                          # *表示所有参与链接的文件
}

```

有了链接脚本，再看demo代码：

```

# 汇编代码

.syntax unified
.cpu cortex-m3
.thumb

.section .isr_vector, "a" # 定义一个段 叫做 .isr_vector
.word 0x20008000 # 定义一个字节长度的内存，内容是 0x2000 8000
.word boot_code # 定义一个字节长度的内存，内容是boot_code地址

.global boot_code # 类似于全局变量，使别的文件也可以找到boot_code
.type boot_code, %funciton # 表示 boot_code 是一个函数 而非数据

boot_code: # 相当于定义了一个函数
    mov r1, #0xA5 # 函数体内容
    b .          # 原地jump，类似于while(1)，让函数卡在这一步方便观察

```

这段代码中，链接脚本指明了.isr_vector放在.text段，起始地址是0x800 0000，这个地址是STM32手册规定的

随后定义了两个word，分别赋值0x20008000、boot_code的地址，这是Cortex-M手册规定的

为什么用FLASH启动，要把启动代码放在0x800 0000？

这和Cortex-M设计的存储器划分有关系

简单来说，Cortex-M规定了两件事：

1. 0x0000 0000 到 0x1FFF FFFF之间的区域是CODE区
2. Cortex-M规定：启动必须从0x0000 0000开始

Flash最大的作用就是提供一种复位的手段，Flash里是要放代码的，这些代码也是要被执行的，Cortex-M的Icode总线只能访问CODE区，所以如果一旦把Flash放到CODE区之外，Flash里的代码就没办法被执行了

那么为什么把Flash放到0x0800 0000，而不是直接放在0x0000 0000？

答案是：把Flash放到0x0800 0000可以获得更灵活的启动方式；

由于Cortex-M已经硬性规定了，启动必须从0x0000 0000开始，如果ST公司把STM32的Flash就放在0x0000 0000上，相当于写死了STM32这块芯片的启动方式，这对实际使用时很不利的，一旦烧写到Flash的代码有问题，很容易导致芯片变砖头且救不回来

ST公司的做法是，把0x0000 0000到0x0800 0000给空出来，这块地址并不对应实际存在的Flash或其他存储器，但这块地址会被映射，在使用flash启动的跳线配置下，0x0000 0000被STM32映射到0x0800 0000，复位后，Cortex-M以为自己是从0x0000 0000开始启动的，但实际上是从0x0800 0000开始启动的；

如果选择了其他的跳线配置，可能0x0000 0000就会被路由到其他的地址，就能实现另一种启动方式；就像电脑的正常启动和BIOS一样，能有效防止芯片变砖之后救不回来